

ICPC Bootcamp Day 3 Summary

Based on `day3_slides.pptx`

Session Summary

Day 3 contrasts greedy algorithms with dynamic programming. A greedy algorithm repeatedly makes the locally best choice, hoping that those choices form a global optimum. The slides present the two properties that make greedy reasoning plausible: the greedy-choice property and optimal substructure. They also warn that greedy solutions are easy to believe and easy to get wrong. If the local choice cannot be justified, the safer next tool is often dynamic programming.

The coin change example makes that warning concrete. With standard Canadian coin denominations, repeatedly taking the largest coin that fits produces an optimal solution for the example amount. With a custom denomination set such as $\{1, 3, 4\}$, the same strategy fails for target 6: greedy chooses $4 + 1 + 1$, while the optimum is $3 + 3$. The lesson is not that greedy is bad, but that greedy needs proof. A correct greedy solution is usually fast and simple; an unproved greedy solution is a common source of wrong answers.

The second half of the deck introduces dynamic programming as exhaustive search with memory. DP works when a problem has optimal substructure and overlapping subproblems. Instead of recomputing the same subproblem many times, DP stores answers by state and reuses them. This often turns an exponential brute-force recursion into a polynomial-time solution. The slides emphasize common problem signals such as "maximize", "minimize", and "count the ways".

The main case study is UVa 11450, *Wedding Shopping*, a variation of knapsack. Given a budget, several garment categories, and model prices for each garment, the goal is to buy one model from each category while spending as much of the budget as possible. Greedy fails because choosing the most expensive current model may leave no valid choice for a later garment. Divide and conquer is also a poor fit because the subproblems are not independent: the remaining budget after one choice constrains every later choice.

The complete-search recurrence uses state (g, b) , where g is the current garment category and b is the remaining budget. From a state, try every model in the current category and recurse to $g + 1$ with the reduced budget. Invalid states return a bad value when the budget becomes negative, and terminal states return the amount spent once all garments have been chosen. The unoptimized search can be enormous because every category may branch across many models.

Top-down DP adds memoization to this recurrence. Since the problem has at most 20 garment categories and 201 possible remaining budgets in the slide's constraints, there are only $20 \times 201 = 4020$ distinct states. If each state tries at most 20 models, the work is roughly 80400 transitions per test case. Bottom-up DP builds a table iteratively from base cases, marking reachable remaining budgets row by row. The deck also notes a memory optimization: if only the previous row is needed, two rows can replace the full table.

Key Takeaways

- Use greedy only when the local choice can be defended.
- DP is useful when recursive search revisits the same state many times.
- A good DP solution starts by naming the state, transitions, base cases, and answer extraction.
- Top-down memoization and bottom-up tabulation are usually equivalent tools, with different implementation tradeoffs.

CP4 Reading Connections

- CP4 Section 3.4, *Greedy*, especially Section 3.4.1 for examples and proof habits.
- CP4 Section 3.5, *Dynamic Programming*, for the Wedding Shopping case study and state-based reasoning.
- CP4 Sections 3.5.1, 3.5.2, and 3.5.3 for DP illustration, classical examples, and non-classical DP patterns.
- CP4 Section 3.2 for the complete-search recurrence that DP optimizes.